ECUE 3.1.3: Neural Communication

From Biological Mechanisms to Artificial Computational Architectures

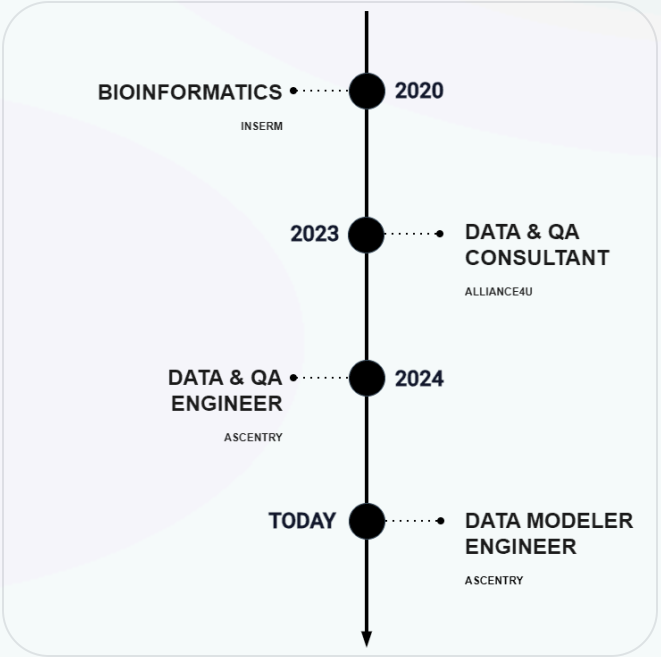
CESI Bachelor AI (Year 2)

Jérémy MARTIN • MSc in Bioinformatics and Systems Biology

Introduction

Who Am I?

- **Background and Research**
  - **MSC IN BIOINFORMATICS & SYSTEMS BIOLOGY (2020)**
  - **3 YEARS AT INSERM**: Data pipelines, Transcriptomics analytics & ML on multiplexed fluorescent imaging.
- **Data Engineering & Medtech**
  - **DATA ENGINEER & MODELER AT ASCENTRY**: Data architecture, regulatory-compliant processes, ML & NLP on customer & marketing data



Ice Breaker: "What Is a Neuron?"

Dynamic Exchanges

- When you hear the word **neuron**, what comes to mind?
- Do you think a neuron is more like:
  - A simple electrical switch (ON/OFF)?
  - A tiny analog computer performing calculus?
  - A bio-chemical factory?
- How many neurons and synapses are there in the human brain?

Key Brain Figures

The brain is **AWESOME**

- $\pm 86 \times 10^9$ neurons *(86 billion)*[*]
- $\pm 10^{14}$ synapses[*]
- Average connectivity: $\approx 7,000$ synaptic inputs per neuron
- Power consumption: ≈ 20 Watts

SOURCE[*]: HTTPS://PMC.NCBI.NLM.NIH.GOV/ARTICLES/PMC11423976/

Course Roadmap

Course will be split into two parts:

Part 1: Foundations

- 🧭 **1.** Introduction & Epistemological Bridge
  - *Scope and core concepts*
- 🦾 **2.** The Biological Neuron
  - *Conceptual challenge: What is a neuron?*
- f_x **3.** From Biology to Mathematics
  - *Abstraction and Information encoding*
- </> **4.** Logical Gates & McCulloch-Pitts Neurons
  - *Live coding: simulation on NumPy*

Part 2: Architectures & Learning

- 🧠 **5.** The First Learning Neuron
  - *From static M&P model to plasticity*
- 📈 **6.** Limits & Evolution
  - *The XOR Crisis*
- 🏗️ **7.** Multi Layer Perceptrons
  - *Non-linearity, feedforward and communication*
- 🚩 **8.** Synthesis & Closure
  - *Closing the Epistemological Bridge*

The Week at a Glance

	Morning	Afternoon
Monday	Ice breaker, the module, the BRIDGE, the projects *(3h)*	**Ch. 2** The Biological Neuron, start of **Ch. 3**, up to the Python warm-up *(3h)*
Tuesday	**Ch. 3** From Biology to Mathematics *(3h30)*	**Ch. 4** Logic gates workshop, start of **Ch. 5**: plasticity *(3h30)*
Wednesday	Projects: autonomous work, questions *(2h)*	**Ch. 5** The First Learning Neuron, start of **Ch. 6** *(3h30)*
Thursday	**Ch. 6** The XOR Crisis, start of **Ch. 7** *(3h30)*	**Ch. 7** and **Ch. 8** Closure *(1h)*, then Projects: autonomous work *(2h)*

> **21h of course + 4h of project work**, in autonomy, with me around for your questions. **Project article + notebook: Friday 20 November. Presentation: Monday 23 November.** Every chapter mixes lecture, **paper & pencil** exercises and **live coding** in Jupyter.

Materials: Everything Lives on GitHub

Repository: [martin-jeremy/cesi-2026-neuronal_communication]

What you will find there *(available now, or added as the course goes)*

- `slides/`: the slides of the course
- `notebooks/`: almost one notebook per chapter
  - `ChapterN_LiveExercises.ipynb`: the exercises
  - `ChapterN_LiveExercises_SOLUTIONS.ipynb`: the solutions
- `projects/`: project subjects, datasets and helpers

Get ready, once

```
git clone <repository-url>
cd <repository>
uv sync                # installs the environment
uv run jupyter lab     # opens Jupyter
```

Then run `notebooks/Chapter1_Setup.ipynb`.

Solutions are there to **check** your work, not to replace it: always try first, on paper, then in code.

Part 1: Foundations • Chapter 1

Introduction & Epistemological Bridge

What This Module IS and IS NOT

Before starting this module, it's important to understand what this course is and is not.

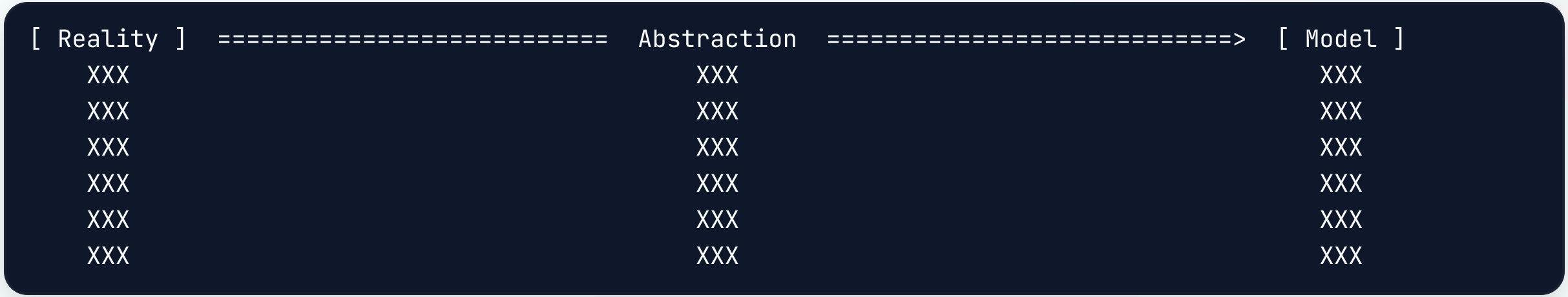
What It Is NOT

- **Not a pure Neuroscience course:** We do not memorize hundreds of anatomical regions, molecular pathways, or clinical pathologies.
- **Not a black-box Deep Learning course:** We will not simply call `import tensorflow` or `torch.nn` to train massive models without inspecting internal mechanics.

What It IS

- **The Formal Bridge:** Connecting biological electrical/chemical communication principles to formal mathematical abstractions.
- **First-Principles AI:** Understanding *why* weights, thresholds, activations, and architectures are designed the way they are.
- **Scratch Implementation:** Building the core computational blocks in pure Python/NumPy.

The Epistemological Bridge



> **Definition:** The **EPISTEMOLOGICAL BRIDGE** is a conceptual framework that maps validated knowledge from one domain onto another.

The Epistemological Bridge

The Map Example

A map is a good way to picture what an epistemological bridge does.

This is a satellite view:



This is a **REALITY**

Each element can be **any** representation:

- coordinates
- color
- density

This is an **ABSTRACTION**

This is a roadmap:



This is a **MODEL**

The Epistemological Bridge

The Map Example

The bridge is a map, not a mirror!
An epistemological bridge is not a perfect **lift-and-shift copy**.

It's an **abstractive** representation of the reality translated into a new system of knowledge, one that remains **coherent** and **consistent** within its own **mathematical** and **logical** space.

The Epistemological Bridge

In our case, the epistemological bridge that will be our **RED LINE** during all the course is the mapping between biological reality and computational model.

```
[ Biological Reality ]  ======================  Abstraction  ======================>  [ Computational Model ]
        XXX                                      XXX                                      XXX
        XXX                                      XXX                                      XXX
        XXX                                      XXX                                      XXX
        XXX                                      XXX                                      XXX
        XXX                                      XXX                                      XXX
        XXX                                      XXX                                      XXX
        XXX                                      XXX                                      XXX
        XXX                                      XXX                                      XXX
        XXX                                      XXX                                      XXX
        XXX                                      XXX                                      XXX
        XXX                                      XXX                                      XXX
        XXX                                      XXX                                      XXX
```

> **Objectives:** At the end of this class, you will be able to fill in the missing parts of the mapping and understand where biological inspiration yields powerful algorithms and where artificial engineering diverges for computational efficiency.

Project Briefs: Two Case Studies

You will be split into **two groups**. Both projects are **case studies** of the Epistemological Bridge, through hands-on implementation and analysis.

Project 1: Neural Decoding for a BCI*

- Spike train dataset: biological neurons answering two stimulus classes
- Implement an artificial neuron from scratch (NumPy)
- Train, evaluate, visualize convergence and learned weights
- **Critical analysis:** what the model captures vs. what it discards from the biological signal

BONUS: *what would need to change in the pipeline to account for what was discarded?*

Project 2: MET* & Robotic Prosthesis

- Spike train dataset: biological neurons answering a motor effort test
- Start with a pre-trained model that worked on a previous experiment
- Identify why it doesn't work in this new case, and repair it
- **Critical analysis:** what does each function change biologically and computationally

BONUS: *does it matter where you put activations if you mix them across layers?*

BCI: *Brain Computer Interface*

MET: *Motor Effort Test*

Projects: What You Need, and When You Get It

To succeed in the projects, you need to...	You get it in	When
Read spike trains: rasters, PSTH, firing rates	Ch. 2 and 3	Mon PM, Tue AM
Turn a trial into a vector of numbers (features)	Ch. 3	Mon PM, Tue AM
Code a neuron, then a perceptron, in NumPy	Ch. 3 to 5	Tue, Wed PM
Evaluate a decoder: train / test split, accuracy, standardization	Ch. 5	Wed PM
Handle more than two classes, layers and activations	Ch. 7	Thu
Criticize your model: what is kept, what is discarded	THE BRIDGE	every chapter

Wednesday morning and **Thursday afternoon**: project time, in autonomy, with me around for your questions. **Article + notebook: 20 November. Presentation: 23 November.**

Evaluation modalities: a mini article (4 pages max), a jupyter notebook and a 30-min presentation followed with questions.
Articles and notebooks due-date: Friday 20 Nov. **Presentation:** Monday 23 Nov.

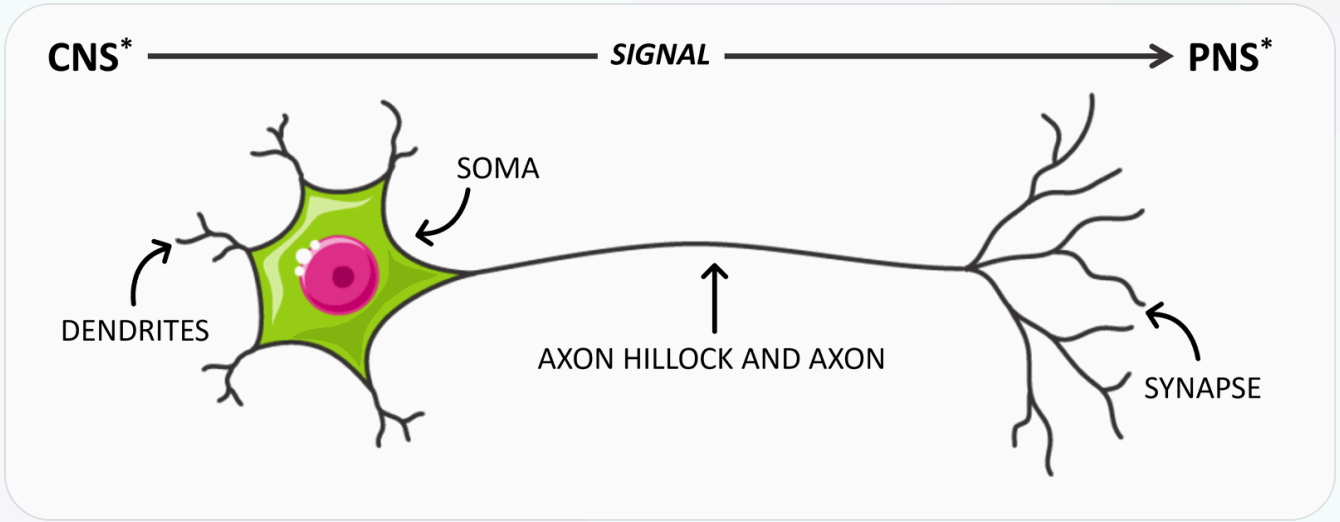
Part 1: Foundations • Chapter 2

The Biological Neuron

Neuron Structure: Overview of a Polarized Anatomy

*Biological neurons are **electrically** excitable cells specialized for receiving, integrating, and transmitting information*

Neuron signaling is **strictly directional** *(polarized)*: information is received by the **dendrites** *(input pole)*, passes through the **soma**, and evaluated at the **axon hillock** before being transmitted along the **axon** to the **synaptic terminals** *(output pole)*.



Some classes of neurons can also transmit information in the opposite direction *(sensory neurons, nociceptors...)*, **but within a single neuron, the signal propagates in one direction only.**

***CNS**: Central Nervous System*

***PNS**: Peripheral Nervous System*

Neuron Structure: Soma & Dendrites

Dendritic Arborization

- Highly branched tree-like extensions often covered with thousands of **dendritic spines**.
- Act as the **input antennas** of the cell.
- Receive chemical neurotransmitters from presynaptic neurons and generate localized postsynaptic potentials (PSPs).

The Soma (Cell Body)

- Metabolic and genetic core containing the nucleus and organelles.
- Acts as a continuous **spatial and temporal integrator**: sums hundreds of incoming graded electrical potentials flowing from dendrites to the axon hillock.

Neuron Structure: Axon & Terminals

Axon & Hillock

- **Axon Hillock:** High density of voltage-gated Na^+ channels; acts as **the decision threshold trigger zone**.
- **Axon:** Long cylindrical projection (up to 1 meter in humans) conducting impulses away from soma.

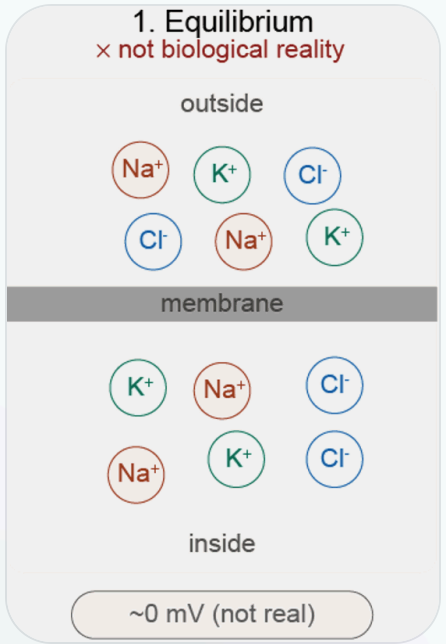
Synaptic Terminals

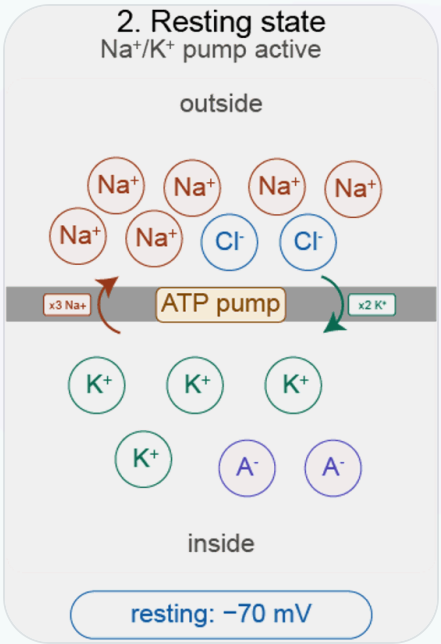
- Specialized terminal boutons containing synaptic vesicles filled with neurotransmitters.
- **Converts electrical pulses into chemical signals** across the synaptic cleft.

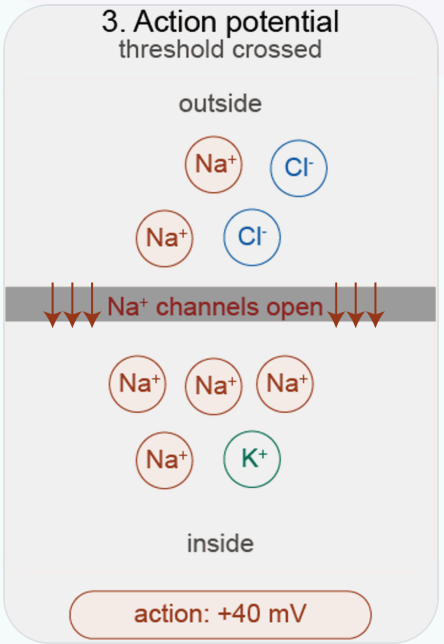
Resting Membrane Potential

But how can a neuron "create" an electrical signal?

→ Thanks to a **gold-standard** engineering pump present in almost ALL our cells!



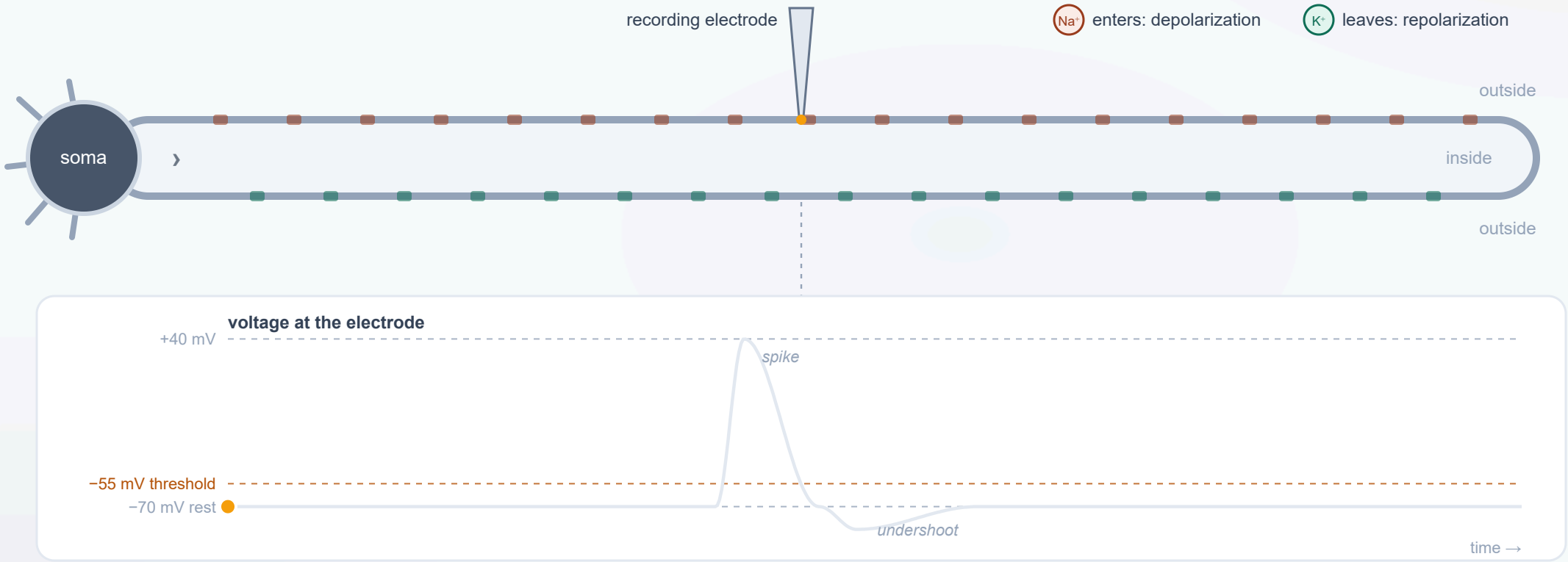




Resting Membrane Potential

But how can a neuron "create" an electrical signal?

→ Thanks to a **gold-standard** engineering pump present in almost ALL our cells!



The Synapse: Where Neurons Communicate

A chemical synapse, step by step

1. The spike reaches the **terminal**
2. **Vesicles** release neurotransmitters into the synaptic cleft
3. They bind **receptors** on the next neuron's dendrite
4. Ion channels open: a small, local potential appears, the **PSP**

Excitatory synapse *(e.g. glutamate)*: an **EPSP**, the membrane moves **toward** the threshold.

Inhibitory synapse *(e.g. GABA)*: an **IPSP**, the membrane moves **away** from the threshold.

Not all synapses are equal: the amount of neurotransmitter and the number of receptors set the **SYNAPTIC STRENGTH**, and this strength can **change** over time.

In Summary:

Neural Communication is nothing more than a Signal Pipeline

1. **Intent**: *An upstream signal reaches the neuron (from the brain or sensory organs).*
2. **Reception**: *Dendrites receive signals and convert them into local electrical shifts.*
3. **Integration & Propagation**: *The soma sums these signals until the threshold is reached at the axon hillock, then an electrical impulse travels down the axon.*
4. **Transmission**: *The axon terminal releases neurotransmitters into the synaptic cleft, initiating a new chemical signal.*

The Epistemological Bridge

We now know the **reality** side. Let's write it down.

```
[ Biological Reality ]  ======================  Abstraction  ======================>  [ Computational Model ]
 Dendritic signals                               XXX                                    XXX
 Synaptic transmission                           XXX                                    XXX
 Resting / Threshold potential                   XXX                                    XXX
 Soma / Membrane sum                             XXX                                    XXX
 Action Potential (all/none)                     XXX                                    XXX
        XXX                                      XXX                                    XXX
        XXX                                      XXX                                    XXX
        XXX                                      XXX                                    XXX
        XXX                                      XXX                                    XXX
        XXX                                      XXX                                    XXX
        XXX                                      XXX                                    XXX
```

> Five mechanisms on the left, nothing on the right yet. **What do we keep, and how do we write it down?** That is Chapter 3.

Part 1: Foundations • Chapter 3

From Biology to Mathematics

Abstraction: What Do We Keep, What Do We Discard?

Remember: **THE BRIDGE IS A MAP, NOT A MIRROR.** Every abstraction choice is a trade-off.

Kept

- **Dendrite → input:** a channel carrying an incoming value
- **Synaptic strength → weight:** how strongly one input counts
- **Somatic summation → weighted sum:** integration of all inputs
- **Threshold → activation function:** a decision rule on the sum

Discarded

- **Temporal dynamics:** exact spike timing, refractory periods
- **Spatial geometry:** dendrite location, cable properties
- **Neurotransmitter identity:** chemical diversity of synapses
- **Energy constraints:** metabolic cost of firing

Encoding Information: The Firing Rate

A single spike carries almost no information on its own: it's all-or-nothing. So how does the brain encode intensity, e.g., "how loud" or "how bright"?

> **Rate coding:** information is encoded in the FREQUENCY of spikes over a time window, not in the shape or size of any single spike.

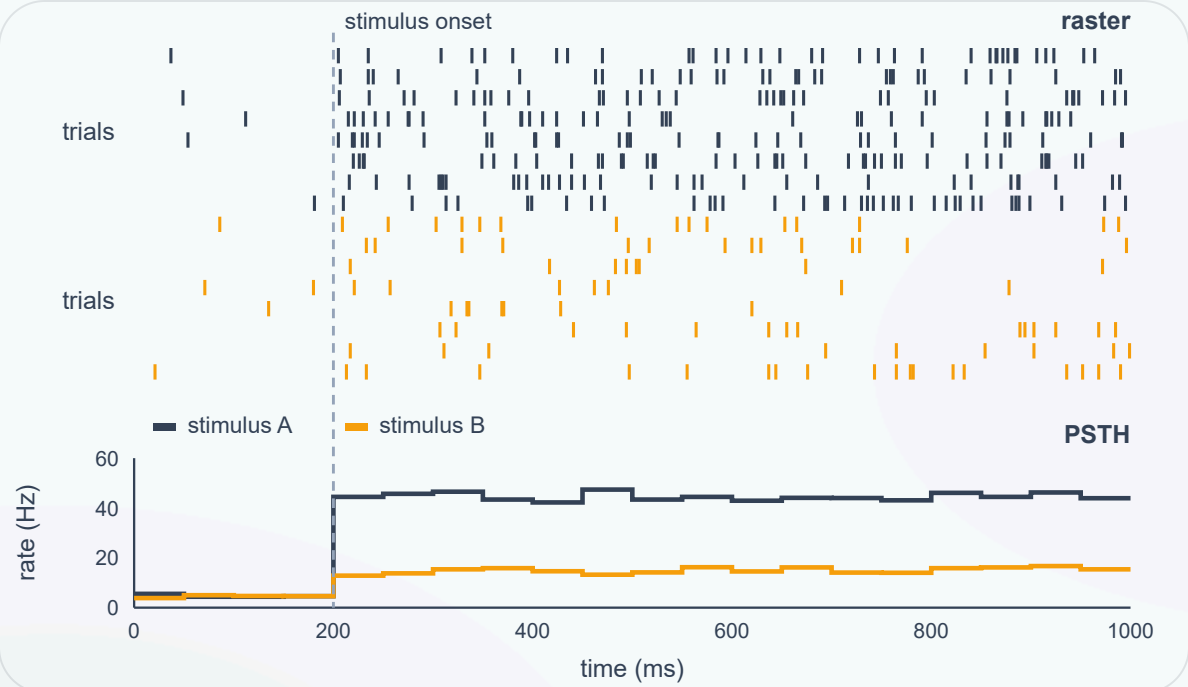
Weak stimulus → few spikes per second

e.g. ~5 Hz

Strong stimulus → many spikes per second

e.g. ~80 Hz

Reading Spike Data: Raster and PSTH



Raster plot: one row per trial, one tick per spike. The same neuron, recorded many times.

PSTH *(peri-stimulus time histogram)*: cut time into bins, count the spikes, average over trials. You get a **firing rate over time**.

> Counting the spikes in a window gives **one number per neuron and per trial**: a firing rate, in Hz.

Rate Is Not the Only Code

Rate coding *(our choice)*

- Information = **how many** spikes in a window
- Simple, robust, easy to compute
- Loses **when** each spike happened
- Needs a **window**: a real-time decoder waits for it to close

Temporal codes *(what we discard)*

- **Latency**: how fast the first spike arrives
- **Timing**: the exact pattern of spikes
- **Synchrony**: neurons firing together

> Choosing rate coding is an **abstraction choice**: two stimuli with the same spike count become **identical** for our model. Keep this in mind for your projects.

TIME TO CODE



notebooks/Chapter3_LiveExercises.ipynb • Part A: Warm-Up

Warm-Up: Simulate a Spike Train

> **</> Check your Python and your maths.** Simulate a neuron, then turn its spikes into a firing rate.

Task: In notebooks/Chapter3_LiveExercises.ipynb, part A:

1. Cut 1 second into **1 ms bins**. In each bin, a neuron firing at r Hz spikes with probability $p = r \times dt$
2. Simulate a **weak** stimulus (5 Hz) and a **strong** one (80 Hz), then **count** the spikes: do you get the rates back?
3. Simulate **3 neurons** at once: an array of shape `(3, 1000)`. Get the vector x of their firing rates
4. Compute $\sum_i w_i x_i$ with $w = (0.5, 0.3, -0.8)$, first with a loop, then with `np.dot`
5. Count the spikes of each neuron in **4 windows of 250 ms**: an array of shape `(3, 4)`

Warm-Up: Simulate a Spike Train

```
rng = np.random.default_rng(184)
dt = 0.001                          # 1 ms bins
n_bins = 1000                       # 1 second

def spike_train(rate, n_bins, dt, rng):
    p = rate * dt                   # spike probability per bin
    return (rng.random(n_bins) < p).astype(int)

weak = spike_train(5, n_bins, dt, rng)
strong = spike_train(80, n_bins, dt, rng)
print(weak.sum(), strong.sum())  # 4 80: about 5 and 80

rates = np.array([40, 20, 10])   # 3 neurons
spikes = (rng.random((3, n_bins)) < rates[:, None] * dt).astype(int)
x = spikes.sum(axis=1) / (n_bins * dt)   # Hz

w = np.array([0.5, 0.3, -0.8])
print(np.dot(w, x))                 # 17.6: close to 18

windows = np.zeros((3, 4), dtype=int)   # 4 windows of 250 ms
for k in range(4):
    windows[:, k] = spikes[:, k*250:(k+1)*250].sum(axis=1)
```

- A spike train is just an array of **0 and 1**
- `sum` over time gives a **count**; divided by the duration, a **rate**
- `axis=1` sums over time, **one rate per neuron**
- `np.dot(w, x)` **is** $\sum_i w_i x_i$
- Counting in **windows** keeps a coarse time axis: each row still adds up to the total

> You just went from a **simulated biological signal** to the **input of an artificial neuron**.

The Mathematical Neuron: The Weighted Sum

We now have everything to write the soma's spatial summation as an equation.

$$z = \sum_{i=1}^{n} w_i x_i$$

- x_i : input i *(firing rate from presynaptic neuron i)*
- w_i : weight of input i *(synaptic strength)*
- z : pre-activation *(total integrated signal at the axon hillock)*

Live Exercise: The Weighted Sum by Hand

✎ **Paper & pencil: no code.** Use the weighted-sum equation $z = \sum_i w_i x_i$ you just derived.

A postsynaptic neuron receives 3 synapses:

Synapse i	x_i *(firing rate, Hz)*	w_i *(synaptic strength)*
1	40	+0.5
2	20	+0.3
3	10	−0.8

Task 1: Compute z for the table above, by hand.

Task 2: Now imagine the 3 presynaptic neurons all go silent: $x_1 = x_2 = x_3 = 0$. Compute z again.

Task 1: $z = 0.5(40) + 0.3(20) - 0.8(10) = 20 + 6 - 8 = 18$

Task 2: with $x_1 = x_2 = x_3 = 0$, the formula gives $z = 0$, **WHATEVER THE WEIGHTS ARE** . Does that match a real neuron at rest?

Introducing the Bias

With only a weighted sum, **zero input always means zero output.** But a biological neuron never rests at 0 mV, it rests at around **-70 mV**, and must climb to a threshold around **-55 mV** before firing.

Bias (b): a constant term, independent of any input, that shifts the pre-activation, playing the role of the **RESTING POTENTIAL / FIRING THRESHOLD OFFSET** .

$$z = \sum_{i=1}^{n} w_i x_i + b$$

From z to a Decision: What's Still Missing

$$z = \sum_{i=1}^{n} w_i x_i + b$$

z is just a **real number**. It can be -7, 0.3, or 42. On its own, it tells us **nothing about whether the neuron fires.**

Remember the biology: firing is **ALL-OR-NOTHING**. A real neuron either sends a spike, or it doesn't. There is no "half a spike."

The Activation Function

> **Activation function (φ):** a function applied to the pre-activation z that produces the neuron's actual output, $y = \varphi(z)$, playing the role of the **FIRING DECISION**.

- **Input:** z, the pre-activation *(can be any real number)*
- **Output:** y, what the neuron actually sends downstream

There are many possible choices for φ; we'll meet others in Chapter 7. For now, we pick the one that matches biology most directly.

The Step Function: Modeling "All-or-Nothing"

$$\varphi(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

$z \geq 0$: the neuron **FIRES** $\rightarrow y = 1$

$z < 0$: the neuron **STAYS SILENT** $\rightarrow y = 0$

This is the **step function** (or Heaviside function): the mathematical counterpart of the firing threshold, now sitting at $z = 0$ thanks to the bias.

The Complete Artificial Neuron

$$z = \sum_{i=1}^{n} w_i x_i + b \qquad\qquad y = \varphi(z)$$

1. **Inputs** x_i: *firing rates from presynaptic neurons*
2. **Weights** w_i: *synaptic strengths*
3. **Integration:** $z = \sum_i w_i x_i + b$ *(spatial summation + resting offset)*
4. **Decision:** $y = \varphi(z)$ *(the firing threshold, as a step function)*

Live Exercise: From z to y (Paper & Pencil)

✎ **Paper & pencil: no code.** Use $z = \sum_i w_i x_i + b$ and the step function φ.

The same postsynaptic neuron as before, now with a bias:

Synapse i	x_i *(Hz)*	w_i
1	40	+0.5
2	20	+0.3
3	10	−0.8

Bias: $b = -25$

Task 1: Compute z, then $y = \varphi(z)$.
Does the neuron fire?

Task 2 (bonus): What is the smallest value of b that would make it fire instead?

Live Exercise: From z to y (Paper & Pencil)

✎ **Paper & pencil: no code.** Use $z = \sum_i w_i x_i + b$ and the step function φ.

The same postsynaptic neuron as before, now with a bias:

Synapse i	x_i *(Hz)*	w_i
1	40	+0.5
2	20	+0.3
3	10	−0.8

Bias: $b = -25$

Task 1:
$z = 18 - 25 = -7 \;\Rightarrow\; y = \varphi(-7) = 0$
the neuron **stays silent**.

Task 2: firing needs $z \geq 0$, i.e.
$18 + b \geq 0 \;\Rightarrow\; b \geq -18.$

TIME TO CODE



notebooks/Chapter3_LiveExercises.ipynb • Part B and Bilan

Live Exercise: From z to y, in NumPy

> **</> Now, the same exercise in code.** Same inputs, same weights, same bias: reproduce Task 1 in NumPy.

Task: In the notebooks/Chapter3_LiveExercises.ipynb, define `x`, `w` and `b` as above, compute `z` with a dot product, implement the step function, and compute `y`. Check it matches your paper answer.

> Don't just copy an answer: **predict y before you run the cell.**

Live Exercise: From z to y, in NumPy

</> Now, the same exercise in code. Same inputs, same weights, same bias: reproduce Task 1 in NumPy.

```python
import numpy as np

x = np.array([40, 20, 10])
w = np.array([0.5, 0.3, -0.8])
b = -25

z = np.dot(w, x) + b        # weighted sum + bias
y = 1 if z ⩾ 0 else 0       # step function

print(z, y)  # -7.0  0
```

Same $z = -7$, same $y = 0$: **the paper computation and the code are the same abstraction**, just two different media.

Chapter Bilan: Rebuild a Neuron From Scratch

> **</> In the notebook, no more scaffolding.** Given a brand new synapse matrix, write the **entire** artificial neuron yourself.

A different postsynaptic neuron, now with 5 synapses:

Synapse i	x_i *(Hz)*	w_i
1	60	+0.4
2	15	+0.2
3	25	−0.6
4	5	+0.9
5	45	−0.3

Bias: $b = -18$

Task: In notebooks/Chapter3_LiveExercises.ipynb, write a single function `neuron(x, w, b)` implementing $y = \varphi\left(\sum_i w_i x_i + b\right)$ from scratch, then apply it to this matrix.

Chapter Bilan: Rebuild a Neuron From Scratch

</> In the notebook, no more scaffolding. Given a brand new synapse matrix, write the **entire** artificial neuron yourself.

```python
def neuron(x, w, b):
    z = np.dot(w, x) + b        # weighted sum + bias
    y = 1 if z ⩾ 0 else 0       # step function
    return y

x = np.array([60, 15, 25, 5, 45])
w = np.array([0.4, 0.2, -0.6, 0.9, -0.3])
b = -18

print(neuron(x, w, b))  # 0
```

$z = -15 \Rightarrow y = 0$: despite mostly excitatory inputs, the inhibitory synapses and the bias keep this neuron **silent**.

The Epistemological Bridge

Let's revisit our red line and fill in what this chapter gave us.

[Biological Reality]	==========	Abstraction	==========>	[Computational Model]
Dendritic signals		Inputs (x)		Feature Vector (x_i)
Synaptic transmission		Efficacy / Weights		Synaptic Weight (w_i)
Resting / Threshold potential		Constant offset		Bias b
Soma / Membrane sum		Linear combination		Affine Sum: z = Σ w_i x_i + b
Action Potential (all/none)		Decision rule		Activation: y = φ(z)
XXX		XXX		XXX
XXX		XXX		XXX
XXX		XXX		XXX
XXX		XXX		XXX
XXX		XXX		XXX
XXX		XXX		XXX

> Five rows filled. The table is **still not complete**: a single neuron is not yet a network, and nothing here **learns** from data. We'll keep filling it in as the course goes.

Part 1: Foundations • Chapter 4

Logical Gates & McCulloch-Pitts Neurons

The McCulloch-Pitts Neuron (MCP, 1943)

> In 1943, neurophysiologist **Warren McCulloch** and logician **Walter Pitts** published the first formal, computable model of a neuron.

- Every input and output is **binary**: 0 or 1
- Integration: $z = \sum_i w_i x_i + b$
- Decision: the **step function**, $y = \varphi(z)$
- No learning: weights are **fixed by hand**
- Their claim: a network of such neurons could compute

  ANY LOGICAL PROPOSITION

Why Logic Gates? From Neurons to Boolean Functions

A **LOGIC GATE** is a function that takes one or more binary inputs and returns a single binary output, following a fixed truth table.

Why gates, specifically?

- Their truth tables are small and fully known
- We can *verify by hand* whether a neuron is correct
- Computers themselves are built from these same gates

The exercise ahead

- Given a truth table, *find* w_i and b by hand
- Then check every row of the table matches $y = \varphi(z)$

Modeling AND with a Neuron

For this part, we will discuss the exercise ahead, and therefore, give a try to represent them in NumPy within notebooks/Chapter4_LiveExercises.ipynb.

x_1	x_2	x_1 AND x_2
0	0	0
0	1	0
1	0	0
1	1	**1**

$$z = w_1 x_1 + w_2 x_2 + b$$

$$y = \varphi(z)$$

Only the **last row** must reach $z \geq 0$; the other three must stay $z < 0$.

Live Exercise: Find the Weights for AND

✎ **Paper & pencil.** Find w_1, w_2 and b so that $y = \varphi(w_1 x_1 + w_2 x_2 + b)$ reproduces the AND truth table.

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

Hint: try $w_1 = w_2 = 1$ first, and search for a b that separates row $(1, 1)$ from the rest.

Live Exercise: Find the Weights for AND

✎ **Paper & pencil.** Find w_1, w_2 and b so that $y = \varphi(w_1 x_1 + w_2 x_2 + b)$ reproduces the AND truth table.

One valid solution: $w_1 = w_2 = 1, b = -1.5$

x_1	x_2	$z = x_1 + x_2 - 1.5$	$y = \varphi(z)$	**AND**
0	0	-1.5	0	0
0	1	-0.5	0	0
1	0	-0.5	0	0
1	1	0.5	**1**	1

Modeling OR with a Neuron

x_1	x_2	x_1 OR x_2
0	0	0
0	1	**1**
1	0	**1**
1	1	**1**

Same equation, same shape, **DIFFERENT THRESHOLD** .

Only $(0, 0)$ must now stay below zero; the other three must fire.

Live Exercise: Find the Weights for OR (and NOT)

> ✎ **Paper & pencil, on your own.** Find weights and a bias for OR, then for NOT.

OR

x_1	x_2	OR
0	0	0
0	1	1
1	0	1
1	1	1

NOT *(single input)*

x	NOT x
0	1
1	0

Live Exercise: Find the Weights for OR (and NOT)

OR: $w_1 = w_2 = 1, b = -0.5$

x_1	x_2	z	y
0	0	-0.5	0
0	1	0.5	1
1	0	0.5	1
1	1	1.5	1

NOT: $w = -1, b = 0.5$

x	$z = -x + 0.5$	y
0	0.5	1
1	-0.5	0

Notice the **sign** of w in NOT: a **negative weight** is exactly what an inhibitory synapse looks like in this model.

Limits of a Single Neuron: A First Hint

> One more gate: **XOR** ("exclusive or") fires when **exactly one** input is active, not zero, not both.

x_1	x_2	XOR
0	0	0
0	1	**1**
1	0	**1**
1	1	0

Try it: spend two minutes searching for w_1, w_2, b. Don't worry if you don't find one.

> If you didn't find a solution, **that's expected.** Keep this table in mind: we will come back to it, by name, very soon.

TIME TO CODE



notebooks/Chapter4_LiveExercises.ipynb

Live Coding: Simulating McCulloch-Pitts Neurons in NumPy

> </> **Let's implement one generic neuron** and test it against every gate we just solved by hand.

Task: In notebooks/Chapter4_LiveExercises.ipynb:

1. Write `mp_neuron(x, w, b)`: weighted sum, bias, step function
2. Plug in the (w, b) you found on paper for **AND**, **OR** and **NOT**: every row must match the truth table
3. Try your own (w_1, w_2, b) for **XOR**
4. *Bonus*: let the machine search a whole grid of (w_1, w_2, b) for XOR

> The function is **the same** for every gate: only w and b change.

Live Coding: Simulating McCulloch-Pitts Neurons in NumPy

```python
import numpy as np

def mp_neuron(x, w, b):
    z = np.dot(w, x) + b
    return 1 if z ⩾ 0 else 0

AND = dict(w=[1, 1], b=-1.5)
OR  = dict(w=[1, 1], b=-0.5)
NOT = dict(w=[-1],   b=0.5)

for x1 in (0, 1):
    for x2 in (0, 1):
        x = [x1, x2]
        print(x1, x2, mp_neuron(x, **AND), mp_neuron(x, **OR))
```

- Every printed row matches the truth tables: **computed**, not guessed
- XOR: every (w_1, w_2, b) fails on at least one row
- Bonus: a grid of 25^3 triples, **none** reproduces XOR

> Two failures now, by hand and by brute force. Coincidence, or a real limit? **Part 2** answers.

The Epistemological Bridge

This chapter used our neuron for something new. What?

[Biological Reality]	==========	Abstraction	==========>	[Computational Model]
Dendritic signals		Inputs (x)		Feature Vector (x_i)
Synaptic transmission		Efficacy / Weights		Synaptic Weight (w_i)
Resting / Threshold potential		Constant offset		Bias b
Soma / Membrane sum		Linear combination		Affine Sum: z = Σ w_i x_i + b
Action Potential (all/none)		Decision rule		Activation: y = φ(z)
Firing or silence		Truth value (1 / 0)		Logic gate: MCP neuron (1943)
XXX		XXX		XXX
XXX		XXX		XXX
XXX		XXX		XXX
XXX		XXX		XXX
XXX		XXX		XXX

> Where we **diverged**: inputs became **binary**, the firing rate is gone. Weights are still **hand-picked**, and XOR is still out of reach.

Part 1 Closure: What We've Built

From biology to a working neuron

- Signal pipeline → weighted sum + bias + step
- One neuron, by hand, **can** compute AND, OR, NOT

Still open

- XOR resists a single neuron: **xxx**
- Weights so far were **hand-picked**, never learned: **xxx**

Part 2 picks up exactly here: **The First Learning Neuron**, where weights stop being guessed and start being *learned* from data.

Part 2

Architectures & Learning

Part 2: Architectures & Learning • Chapter 5

The First Learning Neuron

Where We Left Off: Hand-Picked Weights

What we did for AND, OR, NOT

- Read the truth table
- Reason about the rows
- Try a (w_1, w_2, b), check, adjust, repeat

What a real neuron faces

- $\approx 7,000$ synaptic inputs per neuron
- No truth table written anywhere
- Only **experience**: examples, one after the other

Question: could you hand-pick **7,000 weights** the way we picked 3?

Hand-tuning **does not scale**, and it is not what brains do.

We need a **RULE** that lets the neuron **adjust its own weights** from examples.

Biology: Synapses Change With Experience

> *"When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased."*
>
> Donald Hebb, *The Organization of Behavior*, 1949

Synaptic plasticity: the strength of a synapse is **not fixed**. It changes with the activity that flows through it.

Evidence (1973): Bliss & Lømo measure **LONG-TERM POTENTIATION** : repeated stimulation durably strengthens hippocampal synapses.

From Hebb to a Learning Rule

> **Hebb's idea, as a rule:** if input x_i is active **and** the neuron fires, strengthen w_i.

Let's try it on AND. Start from $w_1 = w_2 = 0, b = 0$: with our step function, $z = 0$ and the neuron **fires for every input**.

Question: which weights will Hebb's rule strengthen? Will the neuron ever learn to stay silent on $(0, 1)$?

> Hebb's rule only **reinforces what the neuron already does**. The neuron fires on $(0, 1)$, so w_2 grows, so it fires on $(0, 1)$ even more. It never learns AND.

> Missing ingredient: a **TEACHER SIGNAL** . For each example, what the neuron **should** have answered: the **target** t.

Rosenblatt's Perceptron (1958)

> In 1958, psychologist **Frank Rosenblatt** kept the McCulloch-Pitts neuron and added what Hebb was missing: a rule that **learns from its mistakes** on labelled examples.

Same neuron as Part 1

- Integration: $z = \sum_i w_i x_i + b$
- Decision: the **step function**, $y = \varphi(z)$

What's new

- Examples come with a **target** t
- Weights are **updated** after each mistake
- Built in hardware: the *Mark I Perceptron*, 400 photocells, weights on motor-driven potentiometers

Learning From Mistakes

For each example (x, t), the neuron answers $y = \varphi(z)$, and we compare:

$$e = t - y$$

Question: with binary t and y, what values can e take? And what should we do to z in each case?

t	y	$e = t - y$	**What happened**	**What to do**
0	0	0	Correct	**NOTHING**
1	1	0	Correct	**NOTHING**
1	0	$+1$	Silent, should have fired	**raise** z
0	1	-1	Fired, should have stayed silent	**lower** z

The Perceptron Learning Rule

$$w_i \leftarrow w_i + \eta\,(t - y)\,x_i \qquad\qquad b \leftarrow b + \eta\,(t - y)$$

- t : target *(what the neuron should answer)*
- y : output *(what it actually answered)*
- $t - y$: error *(its sign gives the direction)*
- x_i : input i *(only active inputs change)*
- η : learning rate *(size of each correction)*

When the neuron is right, $t - y = 0$: **nothing moves**. It only learns from its mistakes.

Why Multiply by x_i?

$x_i = 0$: input i was silent. It did not contribute to z, so it is **not blamed**: w_i does not change.

$x_i = 1$: input i was active. It pushed z toward the wrong answer, so w_i **gets corrected**.

The rule is **LOCAL** : each synapse only needs its own input, the neuron's output, and the target. Just like Hebbian plasticity, only **active synapses** change.

The Training Loop

```
initialize w = 0, b = 0

repeat for each epoch:
    for each example (x, t):
        y = step(w · x + b)
        w = w + η (t - y) x
        b = b + η (t - y)
    if no mistakes in this epoch:
        stop
```

- **Epoch**: one full pass over every training example
- The rule is applied **one example at a time**
- Stop when a whole epoch goes by **without a single mistake**

REMEMBER: $z = \sum w_i x_i + b$ $y = \varphi(z) = \{1 \text{if } z \geq 0, 0 \text{ otherwise}\}$

Live Exercise: Train OR by Hand

✎ **Paper & pencil: no code.** Start from $w_1 = w_2 = 0$, $b = 0$, $\eta = 1$. Run **epoch 1** on OR, row by row, updating w and b after each row.

x_1	x_2	t	z	y	$e = t - y$	**new** w_1	**new** w_2	**new** b
0	0	0						
0	1	1						
1	0	1						
1	1	1						

Careful: each row uses the weights left by the row **above** it.

REMEMBER: $z = \sum w_i x_i + b$ $y = \varphi(z) = \{1 \text{if } z \geq 0, 0 \text{ otherwise}\}$

Live Exercise: Train OR by Hand

✎ **Paper & pencil: no code.** Start from $w_1 = w_2 = 0$, $b = 0$, $\eta = 1$. Run **epoch 1** on OR, row by row, updating w and b after each row.

x_1	x_2	t	z	y	$e = t - y$	**new** w_1	**new** w_2	**new** b
0	0	0	0	1	-1	0	0	-1
0	1	1	-1	0	$+1$	0	1	0
1	0	1	0	1	0	0	1	0
1	1	1	1	1	0	0	1	0

Question: epoch 1 is over. **Are we done?**

REMEMBER: $z = \sum w_i x_i + b$ $y = \varphi(z) = \{1 \text{if } z \geq 0, 0 \text{ otherwise}\}$

Live Exercise: Train OR by Hand

✎ **Paper & pencil: no code.** Start from $w_1 = w_2 = 0$, $b = 0$, $\eta = 1$. Run **epoch 1** on OR, row by row, updating w and b after each row.

Not done: epoch 1 made 2 mistakes, and with $w = (0, 1)$, $b = 0$, row $(0, 0)$ still fires. Keep going.

Epoch	**1**	**2**	**3**	**4**
Mistakes	2	2	1	**0**
(w_1, w_2, b) at the end	$(0, 1, 0)$	$(1, 1, 0)$	$(1, 1, -1)$	$(1, 1, -1)$

After 4 epochs, the neuron found OR **on its own**: $w_1 = w_2 = 1$, $b = -1$.

TIME TO CODE



notebooks/Chapter5_LiveExercises.ipynb

Live Exercise: Training in NumPy

> **</> Now let the machine run the epochs.** Reuse `mp_neuron` from Chapter 4 and wrap it in the training loop.

Task: In notebooks/Chapter5_LiveExercises.ipynb:

1. Write `perceptron_update(x, t, w, b, eta)` and check it reproduces your epoch 1 table
2. Write `train_perceptron(X, T, eta, n_epochs)`, returning `w`, `b` and the number of mistakes per epoch
3. Train it on **OR**, **AND** and **NOT**, and compare the learned weights with the ones you picked by hand
4. Package it as a `Perceptron` class (`fit`, `predict`) that visits the examples in a random order

> Before running AND: **predict** whether it will learn $(1, 1, -1.5)$, like you did.

Live Exercise: Training in NumPy

```python
def perceptron_update(x, t, w, b, eta=1.0):
    y = mp_neuron(x, w, b)
    e = t - y           # error
    w = w + eta * e * x
    b = b + eta * e
    return w, b, y

def train_perceptron(X, T, eta=1.0, n_epochs=20):
    w, b = np.zeros(X.shape[1]), 0.0
    mistakes = []
    for epoch in range(n_epochs):
        errors = 0
        for x, t in zip(X, T):
            w, b, y = perceptron_update(x, t, w, b, eta)
            errors += int(y != t)
        mistakes.append(errors)
        if errors == 0:
            break
    return w, b, mistakes
```

Gate	By hand (w, b)	Learned (w, b)	Epochs
OR	$(1, 1), -0.5$	$(1, 1), -1$	4
AND	$(1, 1), -1.5$	$(2, 1), -3$	6
NOT	$(-1), 0.5$	$(-1), 0$	3

Different numbers, **same truth tables**: many valid answers exist, and the rule finds **one** of them.

NAND: AND, Then Flipped

x_1	x_2	x_1 AND x_2	x_1 NAND x_2
0	0	0	**1**
0	1	0	**1**
1	0	0	**1**
1	1	1	0

- NAND = **NOT AND**: every row of AND, flipped
- The output is 1 **by default**, and only both inputs **together** can silence it
- **Universal**: any logic circuit, a whole processor included, can be built from NAND gates only

> As a neuron: a cell that fires **spontaneously**, and that only the **joint** activity of two inhibitory inputs can silence.

NOR: OR, Then Flipped

x_1	x_2	x_1 OR x_2	x_1 NOR x_2
0	0	0	**1**
0	1	1	0
1	0	1	0
1	1	1	0

- NOR = **NOT OR**: every row of OR, flipped
- The output is 1 **only in silence**: a single active input is enough to switch it off
- **Universal** too: NOR gates alone can also build any logic circuit

> As a neuron: a cell that fires at rest, and that **any** of its inhibitory inputs can silence on its own.

NOT vs NOR: One Input, or Two

NOT: one input, flipped

x	NOT x
0	**1**
1	0

NOR: two inputs, combined by OR, **then** flipped

x_1	x_2	x_1 OR x_2	x_1 NOR x_2
0	0	0	**1**
0	1	1	0
1	0	1	0
1	1	1	0

> NOT flips **one** signal. NOR flips the **OR of several** signals: it outputs 1 only when **every** input is 0.

Question: what happens with **3 signals**, $x_1 \text{ NOR } x_2 \text{ NOR } x_3$?

Feed NOR the Same Input Twice: It Becomes NOT

x	$x_1 = x_2 = x$	$x \text{ NOR } x$	$\text{NOT } x$
0	$(0, 0)$	**1**	**1**
1	$(1, 1)$	0	0

- Only the rows $(0, 0)$ and $(1, 1)$ are used: on them, NOR is exactly NOT
- So NOT is a **special case** of NOR
- And from NOT and NOR you get OR back: $x_1 \text{ OR } x_2 = \text{NOT}(x_1 \text{ NOR } x_2)$

> As neurons: NOT and NOR share one recipe, a cell that **fires at rest** and that **inhibitory** inputs switch off. NOT has one such input, NOR has two.

MAJORITY: A Vote Among Three Inputs

x_1	x_2	x_3	inputs on	MAJORITY
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	2	**1**
1	0	0	1	0
1	0	1	2	**1**
1	1	0	2	**1**
1	1	1	3	**1**

- **Three** inputs, $2^3 = 8$ rows
- The output is 1 when **at least 2** of the 3 inputs are on
- It depends only on **how many** inputs are on, not on **which** ones

> As a neuron: **spatial summation**. One active synapse is not enough; two together reach the threshold.

Live Exercise: New Gates, Nobody Picks the Weights

> ✎ **Paper first, then the notebook.** In Chapter 4, **you** picked the weights. Now the rule does it, on three gates it has never seen.

NAND: 1 everywhere, except $(1, 1)$

NOR: 1 only on $(0, 0)$

MAJORITY: 1 when at least 2 of the 3 inputs are on

Task: In notebooks/Chapter5_LiveExercises.ipynb:

1. On paper: find weights and a bias for **MAJORITY**
2. Write the targets of the three gates and train them with `train_perceptron`
3. Compare the learned MAJORITY neuron with yours: does it treat the three inputs the same way?
4. Compare the learned NOR with the NOT learned just before: what do you notice?

Live Exercise: New Gates, Nobody Picks the Weights

Gate	Learned w	Learned b	Epochs
NAND	$(-2, -1)$	2	6
NOR	$(-1, -1)$	0	4
MAJORITY	$(1, 1, 1)$	-2	4

- **NAND, NOR**: the neuron fires by default ($b \geq 0$), and **inhibitory** weights silence it
- **NOR** $(-1, -1), 0$ is the learned **NOT** $(-1), 0$ with a second, identical inhibitory input
- **MAJORITY**: three equal weights, one **vote** per input, and the neuron fires from 2 votes

> The rule found inhibition and voting **on its own**: nobody wrote these weights.

Think: Can One Neuron Build Every Gate?

> **No code, discuss in pairs.** In Chapter 4, no MCP neuron could reproduce XOR. What about the new gates?

1. Can a single MCP neuron reproduce **NAND**, **NOR** and **MAJORITY**? How do you know, **without** searching for weights?
2. With 2 inputs, a truth table has 4 rows, so there are $2^4 = 16$ possible gates. How many can **one** neuron reproduce?
3. Which ones are **impossible**, like XOR? *Hint*: start from XOR and flip it.

All 16 Gates of Two Inputs

Gate	$(0,0)$	$(0,1)$	$(1,0)$	$(1,1)$	1 neuron?
FALSE *(always 0)*	0	0	0	0	✅
AND	0	0	0	1	✅
x_1 AND NOT x_2	0	0	1	0	✅
x_1 *(copy)*	0	0	1	1	✅
NOT x_1 AND x_2	0	1	0	0	✅
x_2 *(copy)*	0	1	0	1	✅
XOR	**0**	**1**	**1**	**0**	❌
OR	0	1	1	1	✅

Gate	$(0,0)$	$(0,1)$	$(1,0)$	$(1,1)$	1 neuron?
NOR	1	0	0	0	✅
XNOR	**1**	**0**	**0**	**1**	❌
NOT x_2	1	0	1	0	✅
x_1 OR NOT x_2	1	0	1	1	✅
NOT x_1	1	1	0	0	✅
NOT x_1 OR x_2	1	1	0	1	✅
NAND	1	1	1	0	✅
TRUE *(always 1)*	1	1	1	1	✅

14 gates out of 16 fit in one neuron. Only **XOR** and **XNOR** do not.

Think: Can One Neuron Build Every Gate?

- **NAND, NOR, MAJORITY: yes.** The rule stopped on an epoch without a mistake, so valid weights exist
- **2 inputs: 14 gates out of 16** are possible
- **Impossible**: XOR and its flip, **XNOR** ("both equal")

x_1	x_2	XOR	XNOR
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

- With 3 inputs, **PARITY** (1 when an odd number of inputs is on) is impossible too
- The impossible gates all answer the same question: do the inputs **agree**? Not **how many** are on

> One neuron can count active inputs. It cannot tell whether two inputs **agree**.

Why exactly, and how to get around it? **NEXT CHAPTER**

Live Exercise: The Perceptron as a Class

```python
class Perceptron:
    def __init__(self, n_features, eta=1.0, epochs=20, seed=0):
        self.w = np.zeros(n_features)
        self.b = 0.0
        self.eta, self.epochs = eta, epochs
        self.rng = np.random.default_rng(seed)
        self.mistakes_ = []

    def predict(self, X):
        z = X @ self.w + self.b      # one z per row
        return (z >= 0).astype(int)  # step on all rows

    def fit(self, X, T):
        for epoch in range(self.epochs):
            errors = 0
            for k in self.rng.permutation(len(X)):
                self.w, self.b, y = perceptron_update(
                    X[k], T[k], self.w, self.b, self.eta)
                errors += int(y != T[k])
            self.mistakes_.append(errors)
            if errors == 0:
                break
        return self
```

Attributes (`self.w`, `self.b`, `self.mistakes_`): the state of *this* neuron. **Methods** (`fit`, `predict`): what it can do. `self` is the neuron itself.

```python
p = Perceptron(n_features=2).fit(X, T)
p.predict(X), p.w, p.mistakes_
```

Gate	Learned (w, b), random order	Epochs
OR	$(1, 1), -1$	5
AND	$(1, 1), -2$	7
NOT	$(-1), 0$	3

Same rule, same truth tables. A new order of the examples, a new solution: AND is symmetric again.

Live Exercise: One Problem, Many Solutions

> **</> In the notebook:** the `seed` changes the order of the examples. Does it change the answer?

Task: In notebooks/Chapter5_LiveExercises.ipynb:

1. Train a `Perceptron` on **AND** with each seed from 0 to 9
2. Keep each **different** (w_1, w_2, b) in a list: how many are there? Do they all compute AND?
3. For each solution, and for your Chapter 4 weights $(1, 1)$, -1.5: print z on the 4 inputs

> A noisy input arrives: $(0.9, 1)$ instead of $(1, 1)$. **Which neuron would you trust?**

Live Exercise: One Problem, Many Solutions

Neuron	z on $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$	y for $(0.9, 1)$
learned $(1,1)$, -2	$-2, -1, -1, \mathbf{0}$	0
learned $(1,2)$, -3	$-3, -1, -2, \mathbf{0}$	0
by hand $(1,1)$, -1.5	$-1.5, -0.5, -0.5, 0.5$	1

- 10 seeds, **2 solutions**, both compute AND
- Both learned lines pass **exactly on** $(1,1)$: $z = 0$
- The hand-picked neuron keeps a **margin** of 0.5 on every example

> The perceptron stops as soon as it is **right**, not when it is **safe**.

The Perceptron Convergence Theorem

> **Theorem** (Rosenblatt; proof by Novikoff, 1962): **if** some weights and bias classify every training example correctly, the perceptron rule is **guaranteed** to find such weights after a finite number of updates.

What it promises

- No luck needed, no clever starting point
- It always stops, with zero mistakes

What it does not promise

- How many epochs it will take
- Anything at all when **no** such weights exist

Question: which gate do we already know has **no** valid (w_1, w_2, b)?

And XOR?

> **</> In the notebook:** run `train_perceptron` on the XOR truth table for 20 epochs and print the mistakes per epoch.

```
mistakes per epoch: [3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
```

> The mistakes **never reach 0**. The rule is not broken: it keeps hunting for weights that **do not exist**, and the theorem's "if" is not satisfied.

Why does XOR resist a single neuron, and what did it cost the field? **NEXT CHAPTER**

Evaluating a Decoder: Train, Test, Accuracy

Split the trials before learning anything

- **Training set**: the neuron learns on it
- **Test set**: never seen during learning, used **only** to measure

Accuracy $= \dfrac{\text{correct predictions}}{\text{all predictions}}$, on the **test set**

Confusion matrix: where are the mistakes?

	predicted A	**predicted B**
true A	18	2
true B	5	15

Accuracy $= (18 + 15)/40 = 82.5\,\%$, and B is confused with A more often than the reverse.

A score measured on the **training set** says nothing about new trials: the neuron may simply have **memorized** them.

Real Recordings: Standardize Your Features

The problem

- Neurons fire on **different scales**: 2 Hz for one, 80 Hz for another
- A longer recording gives **bigger counts** for the same neuron
- Big numbers dominate the weighted sum, and slow down learning

The fix: z-score each neuron *(each column)*

$$x_i' = \frac{x_i - \mu_i}{\sigma_i}$$

- μ_i, σ_i: mean and standard deviation of neuron i, computed on the **training set**
- A **silent** neuron has $\sigma_i = 0$: drop it, or add a tiny ε

> μ and σ are **part of the model**: every new trial must go through the **same** transformation, and must come from a **comparable** recording.

TIME TO CODE



notebooks/Chapter5_LiveExercises.ipynb • Bilan

Chapter Bilan: Decode a Stimulus From Firing Rates

> **</> In the notebook, no more scaffolding.** Three recorded neurons, two stimuli: train a perceptron to tell which stimulus was shown.

Trial	x_1 *(Hz)*	x_2 *(Hz)*	x_3 *(Hz)*	Stimulus t
1	12	35	20	A *(0)*
2	8	40	25	A *(0)*
3	15	30	18	A *(0)*
4	10	38	22	A *(0)*
5	30	15	21	B *(1)*
6	35	10	19	B *(1)*
7	28	18	24	B *(1)*
8	33	12	20	B *(1)*

Task: In notebooks/Chapter5_LiveExercises.ipynb:

1. Train `train_perceptron` on the 8 trials
2. Predict the stimulus for 4 **new** trials it has never seen
3. Their true stimuli are A, B, A, B: compute the **accuracy** and the **confusion matrix**
4. Plot two **figures**: the mistakes per epoch, and the learned weights
5. Which neuron **matters most**, and which one **carries no information**?

Chapter Bilan: Decode a Stimulus From Firing Rates

</> In the notebook, no more scaffolding. Three recorded neurons, two stimuli: train a perceptron to tell which stimulus was shown.

```
w, b, mistakes = train_perceptron(X_train, T_train)
print(w, b, mistakes)
# [ 18. -20.   1.]  0.0  [2, 0]

X_new = np.array([[11, 33, 23], [31, 14, 22],
                  [14, 36, 19], [26, 20, 18]])
print([mp_neuron(x, w, b) for x in X_new])
# [0, 1, 0, 1]  →  A, B, A, B
```

- **Test**: 4 / 4, accuracy **100 %**, diagonal confusion matrix *(on 4 trials, one mistake costs 25 %)*
- **Figures**: mistakes $2 \to 0$ in 2 epochs; one bar per neuron
- **Neuron 1**: $w_1 = +18$, fires more for B → **excitatory** evidence for B
- **Neuron 2**: $w_2 = -20$, fires more for A → **inhibitory** evidence for B
- **Neuron 3**: $w_3 = +1$, same rate for both → **almost ignored**

The learned weights **describe the biology**: they tell us which neurons encode the stimulus.

Bonus: When the Learning Rate Matters

> </> **Notebook, Bonus 3.** Starting from zero, η only rescales the weights. Now start from a neuron with a **wrong** opinion: $w = (-1, 1, 0)$, $b = 0$, a neuron that prefers A.

Task: In notebooks/Chapter5_LiveExercises.ipynb:

1. Write `train_from(X, T, w0, b0, eta, n_epochs)`: `train_perceptron`, starting from `w0` and `b0`
2. Train the bilan decoder from $(-1, 1, 0)$ with $\eta = 1, 0.1$ and 0.01
3. Compare the learned weights, the epochs, and the test accuracy on the 4 new trials

Bonus: When the Learning Rate Matters

η	Learned w	Epochs	Test accuracy
1	$(17, -19, 1)$	2	100 %
0.1	$(0.8, -1, 0.1)$	2	100 %
0.01	$(0.4, -0.54, -0.01)$	6	**75 %**

- Learned weights = **starting point + sum of the corrections**, and η sets the size of each correction
- Small η: many small steps to undo the wrong start, and a line that is only **just** right

> As soon as the weights do not start at zero, η **changes what the neuron learns**.

Going Further: One Wrong Label

> **</> In the notebook:** trial 4 was an A, but the log says B. On a real experiment, it happens.

Task: In notebooks/Chapter5_LiveExercises.ipynb:

1. Train `train_perceptron` for 50 epochs on the noisy labels: does it stop? Test accuracy?
2. Write `train_pocket`: after each epoch, count the mistakes on the **whole** training set, and keep the **best** weights seen so far "in your pocket"
3. Decode the 4 new trials with the pocket weights

Going Further: One Wrong Label

Plain perceptron, 50 epochs

- Mistakes: $1, 3, 1, 3, \ldots$ **never 0**
- Final weights: every new trial decoded B, **50 %**

Pocket

- Best epoch: **1 mistake**, on trial 4 itself
- New trials: **75 %**
- But neuron 3, which carries no information, gets the **largest** weight

One bad label is enough: the data are **not separable** anymore, like XOR, but by accident.

When the mistakes never reach 0, the final weights depend on **when you stop**: keep the best ones.

Going Further: From Spike Trains to a Decoder

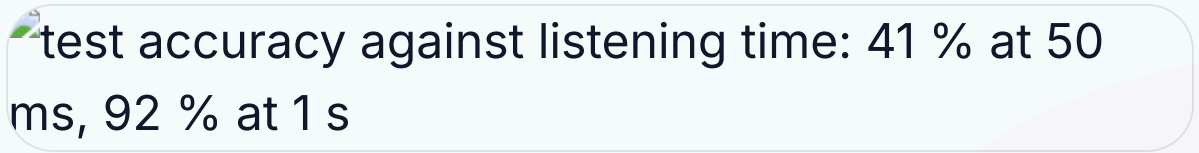
> **</> In the notebook:** no more given rates. Five neurons are recorded as **spikes**, and you count them, as in the Chapter 3 warm-up.

Neuron	1	2	3	4	5
Mean rate for A *(Hz)*	20	35	15	30	25
Mean rate for B *(Hz)*	30	25	15	30	25

Task: In notebooks/Chapter5_LiveExercises.ipynb:

1. Before coding: which neurons should the decoder use?
2. Trials of 1 s: standardize, train a `Perceptron`, test accuracy and confusion matrix
3. Plot the learned weights: does the decoder agree with you?
4. **How long must it listen?** Test accuracy for trials from 50 ms to 2 s

Going Further: From Spike Trains to a Decoder



- **1 s trials**: 95 % on the test trials; largest weights on neurons **1** $(+)$ and **2** $(-)$
- The mistakes **never reach 0**: noisy counts overlap, so neurons 3 to 5 keep small non-zero weights
- **50 ms**: a handful of spikes, one more or less moves the rate by 20 Hz: chance level

Listen longer and decode better, or answer faster and make more mistakes: the trade-off of every **brain-computer interface**.

The Epistemological Bridge

Let's revisit our red line and fill in what this chapter gave us.

[Biological Reality]	Abstraction	[Computational Model]
Dendritic signals	Inputs (x)	Feature Vector (x_i)
Synaptic transmission	Efficacy / Weights	Synaptic Weight (w_i)
Resting / Threshold potential	Constant offset	Bias b
Soma / Membrane sum	Linear combination	Affine Sum: z = Σ w_i x_i + b
Action Potential (all/none)	Decision rule	Activation: y = φ(z)
Firing or silence	Truth value (1 / 0)	Logic gate: MCP neuron (1943)
Synaptic plasticity	Error-driven adaptation	Learning rule: w ← w + η(t-y)x
XXX	XXX	XXX
XXX	XXX	XXX
XXX	XXX	XXX
XXX	XXX	XXX

Question: *which biological mechanism did this chapter translate, and into what?*

> Seven rows filled. Where we **diverged** from biology: the target t, an explicit teacher that Hebb's rule does not have. Still open: XOR, and a **network** of neurons.

Chapter Closure: What the Neuron Learned

From fixed to learned weights

- Synaptic plasticity → the perceptron rule
- The neuron found AND, OR, NOT **on its own**
- Learned weights can be **read back** as biology

Still open

- XOR: neither we nor the rule can find it:
  xxx
- One neuron is not a network: **xxx**

Next chapter: **Limits & Evolution**. Why XOR resists, what it cost the field, and how a **second layer** of neurons gets around it.